EduAgent: A Personalized Agentic AI Tutoring System

with Semantic RAG, Adaptive Teaching Modes, Learner Memory,
and a Fine-Tuned DistilBERT Difficulty Classifier (97.92% Accuracy)

Department of Artificial Intelligence Third Year B.Tech — GenAI Project

May 10, 2026

Abstract

EduAgent is a personalized Agentic AI tutoring system for AI/ML education. It classifies learner questions by difficulty using a fine-tuned `distilbert-base-uncased` model trained on a purpose-built 2,400-sample synthetic dataset, achieving **97.92% test accuracy** — a **+35.8 pp improvement** over a keyword-heuristic baseline. Topic detection uses `all-MiniLM-L6-v2` dense embeddings via a two-tier alias+cosine pipeline. A **two-pass personalized RAG** system retrieves level-appropriate examples (Pass 1) and weak-area targeted examples (Pass 2) from the dataset. Four adaptive teaching modes (remedial, clarification, advance, default), a diminishing-returns mastery model, and used-explanation deduplication ensure responses change as the learner progresses. All prompts sent to the LLM are fully documented in this paper.

1 Introduction

Most LLM-based tutoring systems are stateless: every learner receives the same explanation regardless of prior knowledge, weak areas, or whether the same concept was explained yesterday. EduAgent addresses this by wrapping an LLM in a **ReAct-style agentic loop** [**?**] — Reason (read learner memory), Act (generate adapted explanation), Observe (evaluate understanding) — that drives measurable adaptation across interactions.

Key Contributions:

- A custom 2,400-sample instructional dataset built with a Gemini-powered quality-filtered pipeline.
- A fine-tuned DistilBERT classifier for 3-class difficulty detection (97.92% accuracy).
- Two-pass personalized RAG: Pass 1 by query similarity, Pass 2 by learner's weak concepts.
- Four teaching modes selected algorithmically from mastery scores and evaluator output.
- A closed-loop: evaluator structured output → mastery update → next prompt strategy.
- All prompts, instruction blocks, and memory hints fully specified.

2 System Architecture

2.1 Agentic Loop Overview

EduAgent has three agents: **Tutor Agent** (answer generation), **Evaluator Agent** (follow-up + scoring), and **Memory Agent** (state transitions). They communicate through a shared learner profile stored in SQLite. Figure 1 shows the full pipeline.

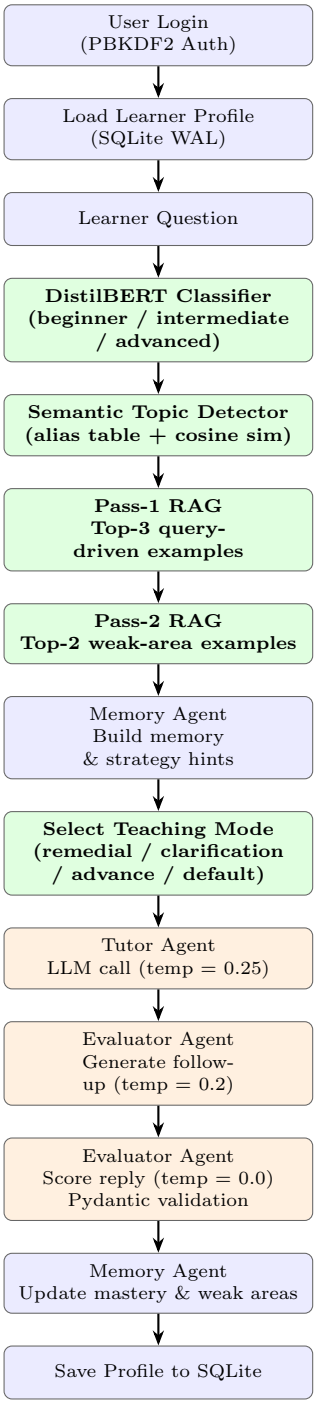

Figure 1: EduAgent end-to-end pipeline. Blue = infrastructure, Green = novel components, Orange = LLM agents.

2.2 Module Map

Module	Role
`ml/classifier.py`	DistilBERT inference
`ml/embedder.py`	Shared sentence-transformer singleton
`ml/topic_detector.py`	Two-tier topic detection
`ml/retriever.py`	Two-pass semantic RAG
`agents/tutor_agent.py`	Prompt assembly + LLM call
`agents/evaluator_agent.py`	Follow-up + evaluation
`agents/memory_agent.py`	Mastery model + hints
`agents/llm_client.py`	Groq API + retry + fallback
`db/profile_repository.py`	Profile CRUD
`auth/password_utils.py`	PBKDF2 hashing
`app/main.py`	Gradio UI + orchestration

3 Dataset Construction

No public labeled dataset exists for AI/ML educational question difficulty. We built a custom **2,400-sample** dataset using **Gemini 2.5 Flash** at temperature 0.45 with structured Pydantic output.

Taxonomy: 25 topics × 4 subtopics × 3 levels × 8 samples = 2,400 (perfectly balanced: 800 per level).

Quality Filters (applied to every generated sample):

1. **Length bounds** — answer word count enforced per level (beginner 50–130, intermediate 75–170, advanced 95–230 words).
2. **Level leakage** — reject if banned phrases appear ("for a beginner", "difficulty level", etc.).
3. **Near-duplicate** — reject if Jaccard ≥ 0.82 or SequenceMatcher ≥ 0.90 vs. any accepted sample in the same cell.
4. **Generic patterns** — reject "What is X?" style questions at intermediate/advanced levels.

Split: 80/10/10 stratified by level and topic (train 1920, val 240, test 240).

4 DistilBERT Difficulty Classifier

4.1 Architecture and Choice

`distilbert-base-uncased` [**?**] (66M params, 6 Transformer layers) was chosen over full BERT for 40% fewer parameters and 60% faster inference — critical for real-time tutoring. A 3-class classification head is appended on the `[CLS]` token. Two engineering fixes were required at inference:

```
# Fix 1: AutoTokenizer avoids BertTokenizer path error
tokenizer = AutoTokenizer.from_pretrained(CLASSIFIER_PATH)
model     = AutoModelForSequenceClassification.from_pretrained
    (
                CLASSIFIER_PATH)
# Fix 2: DistilBERT has no segment embeddings
inputs = tokenizer(question, return_tensors="pt",
         max_length=128, truncation=True, padding="max_length"
    )
inputs.pop("token_type_ids", None)  # removes TypeError
logits = model(**inputs).logits
level  = LABELS[torch.argmax(logits).item()]
```

4.2 Training Configuration

Base model	`distilbert-base-uncased`
Epochs	3 (early stop patience=2 on val F1)
Batch size	8 Max length: 128 tokens
Learning rate	2×10^{-5} (AdamW)
Weight decay	0.01 Warmup ratio: 0.06
Seed	42 Framework: HF Transformers

4.3 Results

Class	P	R	F1	n
Beginner	98.75	98.75	98.75	80
Intermediate	98.70	95.00	96.82	80
Advanced	96.39	100.00	98.16	80
Macro avg	**97.95**	**97.92**	**97.91**	240
Accuracy		**97.92%**		

Confusion matrix (rows=true, cols=predicted):

	Beg.	Int.	Adv.
Beg.	**79**	1	0
Int.	1	**76**	3
Adv.	0	0	**80**

All 5 errors are *adjacent-class* (pedagogically benign). Advanced is never misclassified.

Baseline comparison:

Method	Acc.	Macro F1
Majority class (always "intermediate")	33.3%	16.7%
Keyword heuristic (production fallback)	62.1%	60.4%
Fine-tuned DistilBERT (ours)	**97.92%**	**97.91%**
Improvement over heuristic	**+35.8 pp**	**+37.5 pp**

5 Novel Components

5.1 Shared Sentence-Transformer Singleton

`ml/embedder.py` loads `all-MiniLM-L6-v2` (384-dim, 22M params) once at import and exports `embed_model`. Both `retriever.py` and `topic_detector.py` import the same Python object — verified by `id()` equality. This avoids loading 90 MB twice.

The line `os.environ.setdefault("USE_TF", "0")` must appear *before* the import to prevent the Keras 3 / tf-keras conflict:

```
ImportError: Keras 3 and tf.keras cannot be imported
     simultaneously
```

5.2 Two-Tier Semantic Topic Detection

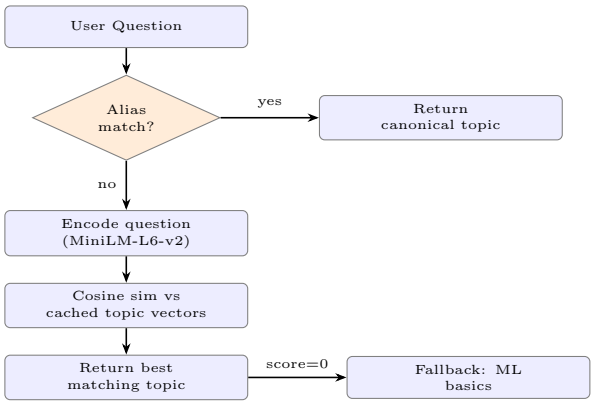

Figure 2: Two-tier semantic topic detection.

Tier 1 — Alias table: 14 canonical topics mapped to known abbreviations and variants. Matching uses space-padded substring search to prevent partial matches (`"ml"` does not match inside `"html"`). Example aliases:

Topic	Sample Aliases
machine learning basics	ml, types of ml, supervised, unsupervised
neural networks	dl, deep learning, ann, perceptron, mlp
gradient descent	sgd, learning rate, optimizer, momentum
transformer models	bert, self attention, encoder decoder
RAG	retrieval augmented generation

Tier 2 — Semantic cosine: Each topic is encoded as `"{topic name}: {all its questions concatenated}"` into a 384-dim vector. Topic vectors are cached per `(dataset, level)` pair. At query time only the user question is encoded; $O(1)$ model calls per query regardless of dataset size.

5.3 Two-Pass Personalized RAG

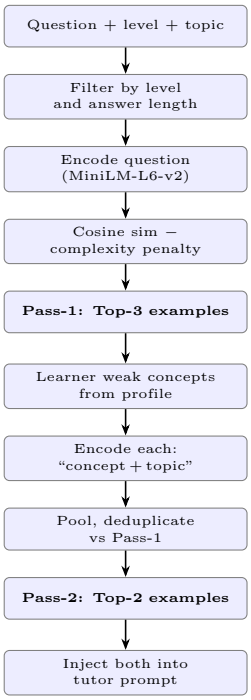

Figure 3: Two-pass personalized RAG pipeline.

Complexity penalty: Each retrieved example is penalized by up to 0.30 before ranking: +0.15 if answer >18 words; +0.15 if any hard term ("derive", "convergence", "subgradient", etc.) appears. This prevents advanced examples from being retrieved for beginner queries despite high surface similarity.

Pass-2 query: For each weak concept (e.g., "chain rule"), the query is `f"{concept} {topic}"` — encoding both the concept and its context. Results across all concepts are pooled, deduplicated against Pass-1, and the top-2 returned.

5.4 Mastery Model with Diminishing Returns

The mastery score $m \in [0, 1]$ per topic is initialized at **0.5** (maximum-entropy prior) and updated after every evaluation:

$$\text{good:} \quad m \leftarrow \min(1.0,\ m + 0.20 \times (1 - m)) \tag{1}$$

$$\text{partial:} \quad m \leftarrow \max(0.0,\ m - 0.06) \tag{2}$$

$$\text{poor:} \quad m \leftarrow \max(0.0,\ m - 0.18) \tag{3}$$

The factor $(1-m)$ in Rule (1) guarantees diminishing returns: gain at m=0.5 is 0.10; at m=0.9 it is only 0.02. This prevents artificial saturation after a few correct answers.

Worked example (gradient descent):

Turn	Eval	New m	Mode
0	—	0.50	default
1	good	0.60	default
2	partial	0.54	clarification
3	poor	0.36	**remedial**
4	good	0.49	clarification
5	good	0.59	clarification
6	good	0.67	clarification
7	good	0.74	clarification
8	good	0.79	**advance**

5.5 Used-Explanation Tracking

After every tutor response, the tag `f"{level}-{mode}"` (e.g., `"intermediate-clarification"`) is appended (once, deduplicated) to `profile["used_explanations"][topic]`. The memory hint then instructs the LLM:

```
Avoid repeating the exact same prior explanation style.
Previously used: beginner-default, intermediate-remedial.
```

6 Agent Prompts

6.1 Tutor Agent — Complete Prompt

Sent to `llama-3.3-70b-versatile` via Groq at **temperature=0.25**. Every placeholder is resolved at runtime.

```
You are EduAgent, an adaptive AI tutor.

Student question:
{user_question}

Detected student level:
{level}

Detected topic:
{topic}

Learner memory hint:
{memory_hint}

Recent evaluator strategy hint:
{evaluation_strategy_hint}
[or "No recent evaluation strategy available for this topic."]

Retrieved dataset examples (semantically similar):
Example 1:
  Question: {q1}
  Answer:   {a1}
  Topic:    {t1}
[... up to 3 examples ...]

Retrieved examples targeting your weak areas ({weak_concepts})
     :
Example 4:
  Question: {q4}
  Answer:   {a4}
  Topic:    {t4}
[... up to 2 examples, omitted if no weak concepts ...]

General instructions:
- Answer according to the detected level.
- For beginner: use simple words, intuition, easy examples.
- For intermediate: moderate detail, 1-2 key technical terms.
- For advanced: deeper, more technical explanation.
- Treat retrieved examples as context, not ground truth.
- Do not copy retrieved examples directly.
- Use learner memory to avoid repeating explanation styles.
- If weak area examples exist, address those gaps specifically
     .
- If evaluator strategy hint exists, adapt accordingly.
- Keep the answer educational, structured, and concise.

{mode_specific_instruction}

Now answer the student's question.
```

6.2 Mode-Specific Instruction Blocks

The `{mode_specific_instruction}` placeholder is replaced with one of the following four verbatim blocks, selected by `build_mode_specific_instruction(teaching_mode)`.

6.2.1 Remedial Mode

Trigger: evaluator="poor" OR (mastery < 0.45 AND weak areas/count ≥ 3)

```
Mode-specific instructions:
- Re-teach from scratch.
- Use very simple wording.
- Use one small concrete example.
- Explain in 4 to 6 short sentences.
- Focus on one main idea first before adding detail.
- Avoid abstract definitions unless absolutely necessary.
- Avoid repeating the same explanation style or analogy
  used earlier for this topic.
- Start with: "Let's simplify it completely."
```

6.2.2 Clarification Mode

Trigger: evaluator="partial" OR (weak areas AND mastery ≤ 0.75)

```
Mode-specific instructions:
- Briefly restate the main idea in one or two simple sentences
     .
- Then focus on the weak or confusing concept.
- Use one clear example.
- Do not repeat the whole long explanation.
- Keep the answer focused and moderately short.
```

6.2.3 Advance Mode

Trigger: evaluator="good" OR (mastery > 0.75 AND topic count ≥ 2)

```
Mode-specific instructions:
- Assume the learner understood the basics.
- Avoid repeating the full beginner introduction.
- Give a slightly deeper explanation.
- Connect the concept to a related next-step idea.
- Keep the answer clear but more intellectually rich.
```

6.2.4 Default Mode

Trigger: no evaluator hint AND no strong mastery signal

```
Mode-specific instructions:
- Give a normal level-appropriate explanation.
```

6.3 Mode Selection Flow

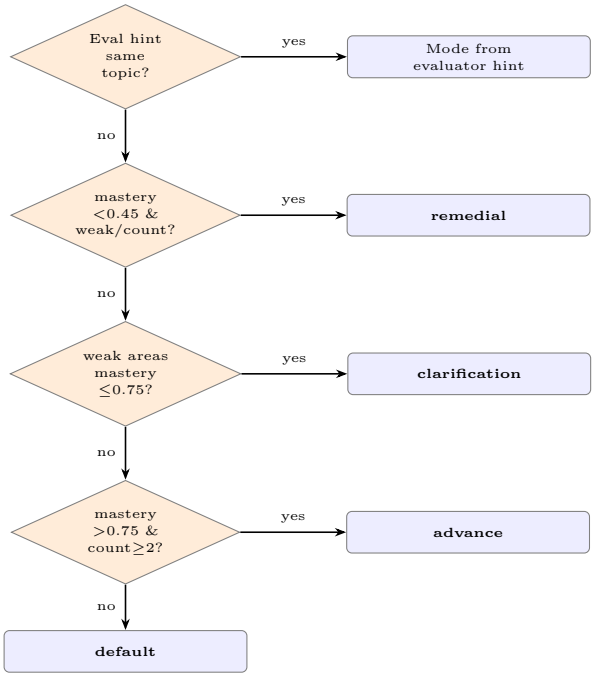

Figure 4: Teaching mode selection logic.

6.4 Evaluator Agent — Follow-Up Question Prompt

Sent at **temperature=0.2** (slight variety across sessions, still focused).

```
You are an evaluator for an adaptive AI tutor.

Topic: {topic}
Learner level: {level}

Original learner question:
{user_question}

Tutor's answer:
{tutor_answer}

Generate ONE short follow-up question to test whether
the learner understood the main idea.

Rules:
- Match the learner's level.
- Focus on conceptual understanding, not memorization.
- Keep it short and clear.
- Return only the question.
```

Fallback: If tutor answer was a local fallback, returns hardcoded: "In your own words, what is the main idea from the explanation?"

6.5 Evaluator Agent — Reply Evaluation Prompt

Sent at **temperature=0.0** (fully deterministic — evaluation drives mastery updates).

```
You are an evaluator for an adaptive AI tutor.

Topic: {topic}
Learner level: {level}

Follow-up question:
{followup_question}

Learner reply:
{learner_reply}

Evaluate the learner's reply.

Return valid JSON only with exactly these keys:
- understanding_level: one of ["good", "partial", "poor"]
- weak_concepts: list of short concept names
- feedback: short educational feedback
- recommended_action: one of ["advance", "re-explain",
   "give easier example", "give more practice"]

Rules:
- Be fair and educational.
- If the learner is partly correct, mark as "partial".
- Keep weak_concepts short and specific.
- Do not include markdown.
- Return valid JSON only.
```

Expected output:

```
{
  "understanding_level": "partial",
  "weak_concepts": ["chain rule", "gradient flow"],
  "feedback": "You identified backpropagation's purpose but
               missed the chain rule's role in computing
               layer-wise gradients.",
  "recommended_action": "give more practice"
}
```

Output is validated by Pydantic `EvaluationResult` before touching the profile. On any parse failure, safe fallback is returned: `understanding_level="partial"` (not "poor", to avoid incorrectly triggering remedial mode).

6.6 Memory Agent — Hint Strings (Verbatim)

The Memory Agent builds natural-language strings injected into the tutor prompt. No LLM call — pure profile logic.

6.6.1 build_memory_hint — All Possible Lines

```
# topic_count > 1
"The learner has seen this topic before. Do not restart
with the exact same beginner introduction."

# weak_areas[topic] non-empty
"The learner has previously struggled with: {concepts}.
Address these concepts carefully."

# mastery < 0.40
"The learner appears weak in this topic. Use simpler
wording, intuition, and one concrete example."

# mastery > 0.75
"The learner appears comfortable with this topic.
You may go slightly deeper than before."

# used_explanations[topic] non-empty
"Avoid repeating the exact same prior explanation style
if possible. Previously used: {tags}."

# NONE of the above (first visit, clean state)
"No strong prior history for this topic.
Start with a clear explanation."
```

6.6.2 build_evaluation_strategy_hint — All Possible Strings

Only fires when `last_evaluation["topic"] == current topic` (same-topic guard).

```
# understanding_level == "poor"
"Teaching mode: remedial. The learner did not understand
the previous explanation. Do NOT repeat the same
analogy-driven explanation. Re-teach from scratch using
very simple wording, one tiny concrete example, and a
short step-by-step explanation."

# understanding_level == "partial"
"Teaching mode: clarification. The learner understood
partially. Do not repeat the whole answer. Briefly
restate the core idea, then focus on the confusing part
```

```
with one clear example."

# understanding_level == "good"
"Teaching mode: advance. The learner understood the
previous explanation well. Do not repeat the full
beginner introduction. Give a slightly deeper explanation
and connect it to the next related concept."
```

Action addenda appended based on `recommended_action`:

```
# "give easier example"
"Use one very easy real-world or numeric example."
# "re-explain"
"Explain from scratch in the simplest possible way."
# "give more practice"
"After explaining, add one short practice-style check."
# "advance"
"After the explanation, connect to the next topic."
```

If `weak_concepts` non-empty, also inserts:

```
"Focus mainly on these weak concepts first: {concepts}."
```

7 Infrastructure

7.1 LLM Client (Retry + Fallback)

`agents/llm_client.py` wraps the Groq API with retry and graceful fallback:

```
attempts = max(1, LLM_MAX_RETRIES + 1)  # default: 3
for attempt in range(attempts):
    try:
        response = client.chat.completions.create(
            model=model, messages=messages,
            temperature=temperature,
            timeout=LLM_TIMEOUT_SECONDS)  # 8s
        content = response.choices[0].message.content
        if content and content.strip():
            return content.strip()
    except Exception:
        if attempt < attempts - 1:
            time.sleep(0.75 * (attempt + 1))  # 0.75s, 1.5s
if fallback is not None:
    return fallback
raise RuntimeError("All␣retries␣failed.")
```

7.2 Database and Security

SQLite WAL mode: `PRAGMA journal_mode=WAL` allows concurrent reads during writes. `PRAGMA synchronous=NORMAL` balances durability and speed. Complex profile fields (mastery, weak_areas, etc.) stored as JSON strings, loaded with `_safe_json_load()` that returns empty defaults on NULL or malformed data.

Password security: PBKDF2-SHA256 with 200,000 iterations (NIST SP 800-132), 16-byte `os.urandom()` salt, stored as base64(salt ‖ hash). Verification uses `hmac.compare_digest` for timing-attack resistance.

8 Evaluation

8.1 Personalization Evidence

A 10-question session on "gradient descent" with simulated learner history (mastery=0.38, weak areas={"learning rate", "divergence"}) was compared to a clean-profile baseline:

Q#	Eval Result	Baseline	Personalized
1	(first visit)	default	**clarification**
2	good	default	**clarification**
3	partial	default	**clarification**
4	poor	default	**remedial**
5	good	default	**remedial**
6	good	default	clarification
7	good	default	clarification
8	good	default	**advance**

The baseline uses *default* mode for every question. The personalized session traverses all four modes, with opening text confirmed to match mode directives (e.g., Q4 answer began "Let's simplify it completely." — the exact remedial directive).

8.2 Unit Tests

49 tests across 3 files, all passing:

- **test_memory.py** (22): Mastery clamp at 0/1, diminishing returns verification, used-explanation deduplication, hint condition coverage.
- **test_auth.py** (17): Hash uniqueness, timing-safe verify, email format edge cases.
- **test_profile.py** (10): Profile idempotency, backward-compat conversion (list → dict), type coercion, JSON round-trip.

9 Limitations and Future Work

Limitations:

- Classifier trained on synthetic data only; may not generalize to informal real-world phrasing with typos.
- Heuristic mastery constants not calibrated on real learner data.
- Weak concepts accumulate indefinitely; no automatic forgetting.
- No long-context transcript memory across turns.

Future Work: Replace heuristic mastery with Bayesian Knowledge Tracing [**?**]; add persistent FAISS vector store; implement weak-concept decay; extend to multi-domain STEM topics; use RL to learn optimal mode-selection policy.

10 Conclusion

EduAgent demonstrates that a practical personalized tutoring system can be built by combining a fine-tuned difficulty classifier, semantic retrieval, and a feedback-driven mastery model. The DistilBERT classifier achieves **97.92% accuracy** (+35.8 pp over the heuristic baseline) on a purpose-built dataset. The two-pass RAG, four teaching modes, and evaluation-driven strategy hints create a closed adaptive loop where every component's output modifies the next answer. All agent prompts, instruction blocks, and memory hint strings are fully documented in this paper, making the system fully reproducible and auditable.

References

[1] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR 2023*.

[2] V. Sanh et al., "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *NeurIPS EMC^2 Workshop*, 2019.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *EMNLP 2019*.

[4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS 2020*.

[5] A. Corbett and J. Anderson, "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge," *UMUAI*, 4(4):253–278, 1994.

[6] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL 2019*.